**Terraform — 3–4 YOE Scenario-Based Q&A**

**1. Terraform state and actual infrastructure are not matching. What will you check?**
First, I compare the Terraform state with the actual infrastructure and identify what is different.
Then I check whether someone manually changed the resource outside Terraform.
I review the Terraform plan and decide whether Terraform should update the resource or the configuration needs to be corrected.
**Simple ga gurthupettuko:**
Terraform state and real infrastructure match avvakapothe → difference enti chudu → manual change jariginda check cheyyi.
Required correction chesi → terraform plan run chesi result verify cheyyi.
**Eng:** Terraform and the real infrastructure are not matching → check what changed → check if someone changed it manually → make the required correction → run the plan again.
**Example:**
terraform plan → Terraform em changes chupisthundo chudu.
terraform state list → resource Terraform state lo unda check cheyyi.
AWS Console → actual resource lo em change ayyindo compare cheyyi.
Difference identify chesi → Terraform code correct cheyyi → terraform plan malli run cheyyi.

---

**2. Terraform state is locked. What will you do?**
First, I check whether another Terraform operation is currently running.
If another run is active, I wait for it to complete instead of removing the lock.
If no run is active, I verify why the lock is still present and remove it only when I am sure it is an old stale lock.
**Simple ga gurthupettuko:**
State lock ayithe → vere Terraform run nadusthundo chudu → run unte wait cheyyi.
Run lekapothe → old lock aa check chesi → safe ani confirm ayithe remove cheyyi.
**Eng:** Terraform state is locked → check if another run is active → wait if it is running → if no run is active, confirm the lock is old and safely remove it.
**Example:**
terraform plan / terraform apply → another Terraform operation running unda chudu.
Jenkins/CI pipeline → Terraform run active ga unda check cheyyi.
Active run lekapothe → stale lock aa verify cheyyi.
Safe ani confirm ayithe → terraform force-unlock LOCK_ID → old lock remove cheyyi.

---

**3. Terraform plan is showing unexpected changes. What will you check?**
First, I check exactly which resources Terraform wants to change.
Then I compare the current Terraform code, variables, state, and actual infrastructure.
I also check whether someone manually changed the infrastructure or whether a module or provider version changed.
**Simple ga gurthupettuko:**
Plan lo unexpected changes vaste → ye resource change avutundo chudu → code, state, variables compare cheyyi.
Manual change or module/provider change unda check chesi → correct issue chesi plan malli run cheyyi.
**Eng:** Unexpected Terraform changes → check which resource is changing → compare code, state, variables, and infrastructure → find the reason → run the plan again.
**Example:**
terraform plan → unexpected change ye resource ki vastundo chudu.
terraform state show RESOURCE_NAME → state lo current value check cheyyi.
Terraform code and variables → expected value tho compare cheyyi.
AWS Console → actual value check cheyyi.
Reason fix chesi → terraform plan malli run cheyyi.

---

**4. Someone manually changed an AWS resource managed by Terraform. What will you do?**
First, I identify what was changed manually and compare it with the Terraform configuration.
Then I run Terraform plan to understand what Terraform wants to change.
If the manual change is not required, I allow Terraform to restore the configuration defined in code.
**Simple ga gurthupettuko:**
Manual change jarigithe → em change chesaro chudu → Terraform code tho compare cheyyi.
terraform plan chesi → required configuration ni Terraform tho restore cheyyi.
**Eng:** Someone changed the resource manually → find what changed → compare it with Terraform code → run the plan → restore the required configuration.
**Example:**
AWS Console → manually changed setting enti chudu.
terraform plan → Terraform aa change ni detect chesthunda check cheyyi.
Terraform code → expected configuration tho compare cheyyi.
Manual change required kakapothe → terraform apply → code prakaram resource restore cheyyi.

---

**5. Terraform apply failed halfway. What will you do?**
First, I check the Terraform output and identify which resource failed.
Then I check whether the resources before the failure were created or modified successfully.
I fix the actual issue and run Terraform plan again before continuing with the apply.
**Simple ga gurthupettuko:**
Apply madhyalo fail ayithe → ye resource daggara fail ayindo chudu → mundu resources successful ga create ayyaya check cheyyi.
Issue fix chesi → plan check chesi → apply continue cheyyi.

**Eng:** Terraform apply failed halfway → find the failed resource → check what was already created → fix the issue → run plan again → continue the apply.
**Example:**
terraform apply → output lo ye resource daggara fail ayyindo chudu.
terraform state list → already created resources state lo unnaya check cheyyi.
Error message → actual issue enti identify cheyyi.
Issue fix chesi → terraform plan → remaining changes check cheyyi.
Plan correct unte → terraform apply → apply continue cheyyi.

---

**6. Terraform resource already exists, but Terraform is trying to create it. What will you check?**
First, I verify whether the resource was created manually or by another Terraform configuration.
Then I check whether the resource is already present in the Terraform state.
If it is not managed by this state, I import it when appropriate instead of creating a duplicate resource.
**Simple ga gurthupettuko:**
Resource already unte → Terraform state lo unda chudu → state lo lekapothe manual ga create ayyinda check cheyyi.
Already existing resource ni required ayithe → Terraform state loki import cheyyi.
**Eng:** The resource already exists → check whether Terraform knows about it → if it is not in the state, import the existing resource instead of creating it again.
**Example:**
AWS Console → resource already exist avutunda chudu.
terraform state list → resource state lo unda check cheyyi.
State lo lekapothe → manual ga create ayyinda verify cheyyi.
terraform import RESOURCE_ADDRESS RESOURCE_ID → existing resource ni state loki import cheyyi.
terraform plan → duplicate creation plan avutunda verify cheyyi.

---

**7. How do you import an existing resource into Terraform?**
First, I create the correct Terraform configuration for that resource.
Then I import the existing resource into the Terraform state using its resource ID.
After importing, I run Terraform plan and make the configuration match the actual resource.
**Simple ga gurthupettuko:**
Existing resource ni Terraform manage cheyyali ante → Terraform code create cheyyi → resource ni state loki import cheyyi.
Tarvatha plan run chesi → code and real resource match cheyyi.
**Eng:** Existing resource needs Terraform management → create its Terraform configuration → import it into the state → run plan → make the code match the real resource.
**Example:**
Terraform code lo resource block create cheyyi.
terraform import aws_instance.web i-123456789 → existing resource ni state loki import cheyyi.
terraform state show aws_instance.web → imported details check cheyyi.
terraform plan → code and real resource match avutunnaya chudu.
Difference unte → code correct chesi plan malli run cheyyi.

---

**8. Terraform is trying to recreate a Production resource. What will you do?**
I do not immediately run terraform apply.
First, I check why Terraform wants to destroy and recreate the resource and whether the change can cause downtime or data loss.
I review the plan carefully and take approval before making a risky Production change.
**Simple ga gurthupettuko:**
Production resource recreate ani plan lo vaste → immediate apply cheyyaku.
Enduku recreate avutundo chudu → downtime/data loss risk check cheyyi → review and approval tarvatha apply cheyyi.
**Eng:** Terraform wants to recreate a Production resource → do not apply immediately → find the reason → check downtime and data loss risk → review and get approval.
**Example:**
terraform plan → -/+ replacement ye resource ki vastundo chudu.
Plan output → destroy and create changes check cheyyi.
AWS Console → resource Production lo important aa verify cheyyi.
Downtime/data loss risk check cheyyi.
Approval tarvatha → terraform apply → change execute cheyyi.

---

**9. Terraform destroy is accidentally planned for Production. What will you do?**
First, I stop the operation and do not approve the destroy.
Then I check the Terraform plan, workspace, variables, backend, and target environment.
I correct the configuration and add stronger Production protection so the same mistake does not happen again.
**Simple ga gurthupettuko:**
Production destroy kanipisthe → immediate ga stop cheyyi → workspace and variables check cheyyi.
Correct environment confirm chesi → protection add cheyyi → appudu matrame apply cheyyi.
**Eng:** Production destroy is planned → stop the operation → verify the environment and configuration → correct the mistake → add protection before applying.
**Example:**
terraform plan → destroy changes unnaya chudu.
terraform workspace show → correct workspace aa check cheyyi.
terraform state list → affected resources identify cheyyi.

Backend and variables → Production environment correct aa verify cheyyi.
Issue correct chesi → terraform plan → unwanted destroy ledu ani verify cheyyi.

---

**10. Terraform backend is not accessible. What will you check?**
First, I check whether the backend storage is available and reachable.
Then I verify credentials, permissions, network connectivity, and backend configuration.
I also check whether the backend service itself has any issue.
**Simple ga gurthupettuko:**
Backend access avvakapothe → backend available ga unda chudu → credentials and permissions check cheyyi.
Network and backend configuration verify chesi → issue fix cheyyi.
**Eng:** Terraform backend is not accessible → check backend availability → verify credentials and permissions → check network and configuration → fix the issue.
**Example:**
terraform init → backend access error em vastundo chudu.
AWS Console → S3 backend bucket available ga unda check cheyyi.
aws sts get-caller-identity → correct credentials use avutunnaya verify cheyyi.
IAM permissions → backend access unda check cheyyi.
Issue fix chesi → terraform init → backend successful ga initialize ayyinda verify cheyyi.

---

**11. Terraform remote state is important in a team. Why?**
Remote state allows multiple team members and CI/CD pipelines to use the same Terraform state.
It also provides centralized state storage and can support state locking depending on the backend.
This prevents different engineers from maintaining separate local state files.
**Simple ga gurthupettuko:**
Team lo Terraform use chesthe → state ni common remote location lo maintain cheyyali.
Andaru same state use chestharu → conflicts and separate local states avoid cheyyachu.
**Eng:** A team uses Terraform → keep the state in a shared remote location → everyone uses the same state → avoid separate local state and conflicts.
**Example:**
terraform init → Terraform remote backend configure ayyinda check cheyyi.
terraform state list → shared state nunchi resources vastunnaya chudu.
Team member A and CI/CD → same backend state use chesthunnara verify cheyyi.
Backend locking → simultaneous changes avoid avutunnaya check cheyyi.

---

**12. Terraform state file was accidentally deleted. What will you do?**
First, I stop further Terraform changes to avoid making the situation worse.
Then I check whether the remote backend has versioning, backups, or previous state versions.
I restore the correct state and verify it carefully before running Terraform plan.
**Simple ga gurthupettuko:**
State delete ayithe → Terraform changes stop cheyyi → backup or previous state check cheyyi.
Correct state restore chesi → verify chesi → appudu plan run cheyyi.
**Eng:** Terraform state was deleted → stop Terraform changes → find a backup or previous state version → restore it → verify the state → run plan.
**Example:**
Terraform operations → first stop cheyyi.
AWS S3 backend → bucket versioning/history check cheyyi.
Previous state version → correct state file identify cheyyi.
State restore chesi → terraform state list → resources correct ga unnaya chudu.
terraform plan → unexpected changes unnaya verify cheyyi.

---

**13. How do you protect Terraform state?**
I store the state in a secure remote backend instead of keeping it only on local machines.
I restrict access to the state because it can contain sensitive information.
I also enable state locking and backup or versioning where supported.
**Simple ga gurthupettuko:**
State ni secure ga maintain cheyyali ante → remote backend use cheyyi → access restrict cheyyi.
Locking and backup/versioning enable chesi → state ni protect cheyyi.
**Eng:** Protect Terraform state → store it in a secure remote backend → restrict access → enable locking and backups or versioning.
**Example:**
terraform init → remote backend configure ayyinda chudu.
AWS IAM → state bucket access required users ki matrame unda check cheyyi.
S3 → versioning/encryption enabled aa verify cheyyi.
Terraform operations → state locking support avutunda check cheyyi.

---

**14. Terraform variables contain sensitive information. How will you manage them?**
I avoid putting passwords, tokens, or secrets directly in Terraform code.
I use secure secret-management solutions or protected CI/CD variables and mark sensitive Terraform variables appropriately.
I also make sure sensitive values are not exposed in logs or source control.
**Simple ga gurthupettuko:**
Password/token ni Terraform code lo rayakudadhu → secure secret storage use cheyyi.
Sensitive values ni protect chesi → Git and logs lo expose avvakunda chudu.

**Eng:** Terraform secrets should not be stored in code → use secure secret storage → protect sensitive values → prevent them from appearing in Git or logs.
**Example:**
Terraform .tf files → password/API key hardcode ayyinda chudu.
Git repository → secret accidentally commit ayyinda check cheyyi.
CI/CD variables or secret manager → secret secure ga store ayyinda verify cheyyi.
Terraform variable → sensitive = true configured aa check cheyyi.
Pipeline logs → secret values print avutunnaya verify cheyyi.

---

**15. Terraform module is causing an unexpected change. What will you check?**
First, I check which module is creating the resource and which values are being passed to it.
Then I compare the current module version with the previous working version.
I also review module variables, defaults, and recent module changes before applying anything.
**Simple ga gurthupettuko:**
Module unexpected change chesthe → ye module ani chudu → values and variables check cheyyi.
Previous version tho compare chesi → recent change identify cheyyi → appudu plan verify cheyyi.
**Eng:** A Terraform module causes an unexpected change → identify the module → check its variables and values → compare with the previous version → find the recent change.
**Example:**
terraform plan → unexpected change ye resource ki vastundo chudu.
Terraform code → aa resource ye module nunchi vastundo check cheyyi.
Module variables → values and defaults verify cheyyi.
Git → previous module version tho compare cheyyi.
Issue fix chesi → terraform plan malli run cheyyi.

---

**16. How do you make Terraform reusable across multiple environments?**
I create reusable Terraform modules and keep environment-specific values outside the common module code.
For example, DEV, QA, and PROD can use the same module with different variables and configuration.
This reduces duplicate Terraform code and keeps the environments consistent.
**Simple ga gurthupettuko:**
DEV, QA, PROD ki same Terraform logic kavali ante → reusable module create cheyyi.
Environment-specific values separate ga petti → same module ni different environments lo use cheyyi.
**Eng:** Multiple environments need the same Terraform logic → create reusable modules → keep environment-specific values separate → use the same module everywhere.
**Example:**
Terraform module → common infrastructure code create cheyyi.
DEV variables → DEV-specific values configure cheyyi.
QA variables → QA-specific values configure cheyyi.
PROD variables → PROD-specific values configure cheyyi.
terraform plan → each environment expected changes verify cheyyi.

---

**17. Terraform configuration has too much duplicate code. What will you do?**
I identify the repeated configuration and move the common logic into reusable modules.
Then I pass environment or application-specific values through variables.
This makes the Terraform code easier to maintain and update.
**Simple ga gurthupettuko:**
Same code repeat avuthe → common code ni module lo move cheyyi → different values variables tho pass cheyyi.
Duplicate code tagguthundi → maintenance easy avutundi.
**Eng:** Terraform code is repeated → move common code into modules → pass different values through variables → reduce duplication and maintenance effort.
**Example:**
Terraform files → repeated resource blocks identify cheyyi.
Common configuration → module lo move cheyyi.
Variables → environment-specific values pass cheyyi.
terraform plan → module expected resources create chesthunda verify cheyyi.

---

**18. Terraform provider version changed and the pipeline started failing. What will you check?**
First, I check the provider version currently being used and compare it with the previous working version.
Then I review the provider changelog and Terraform errors for compatibility or behavior changes.
If required, I use a compatible version and test the configuration before Production.
**Simple ga gurthupettuko:**
Provider update tarvatha fail ayithe → current and old version compare cheyyi.
Error and compatibility check chesi → required version use chesi → test chesi deploy cheyyi.
**Eng:** Provider update caused failure → compare the current and previous versions → check compatibility and errors → use a compatible version → test before Production.
**Example:**
terraform version → Terraform version check cheyyi.
.terraform.lock.hcl → currently selected provider version chudu.
Pipeline logs → exact provider error identify cheyyi.
Provider documentation/changelog → breaking change unda check cheyyi.
Compatible version set chesi → terraform init → terraform plan run cheyyi.

**19. Terraform provider authentication is failing. What will you check?**
First, I check whether the required credentials are available to Terraform.
Then I verify permissions, credential expiry, environment variables, profiles, and the provider configuration.
I also check whether the target cloud account or service is reachable.
**Simple ga gurthupettuko:**
Provider authentication fail ayithe → credentials available aa chudu → expiry and permissions check cheyyi.
Provider configuration and target access verify chesi → issue fix cheyyi.
**Eng:** Provider authentication fails → check credentials → verify expiry and permissions → check provider configuration and target access → fix the issue.
**Example:**
aws sts get-caller-identity → AWS credentials correct ga work avutunnaya chudu.
Environment variables/profile → correct credentials load avutunnaya check cheyyi.
IAM → required permissions unnaya verify cheyyi.
Terraform provider block → configuration correct aa chudu.
Issue fix chesi → terraform plan → authentication successful aa verify cheyyi.

---

**20. Terraform plan works locally but fails in Jenkins. Why?**
I compare the local and Jenkins environments.
I check Terraform version, provider version, credentials, environment variables, backend access, working directory, and available files.
Usually, the issue is caused by an environment or configuration difference rather than Terraform code itself.
**Simple ga gurthupettuko:**
Local lo work ayyi Jenkins lo fail ayithe → rendu environments compare cheyyi.
Terraform version, credentials, variables, backend, and files check chesi → difference fix cheyyi.
**Eng:** Terraform works locally but fails in Jenkins → compare both environments → check versions, credentials, variables, backend, and files → fix the difference.
**Example:**
Local → terraform version → version check cheyyi.
Jenkins → terraform version → same version aa compare cheyyi.
Jenkins credentials → correct AWS credentials unnaya check cheyyi.
Jenkins workspace → required .tf files unnaya chudu.
terraform init → backend access check cheyyi.
Difference fix chesi → terraform plan → Jenkins lo malli run cheyyi.

---

**21. Terraform apply is taking too long. What will you check?**
First, I identify which resource or module is taking most of the time.
Then I check whether the cloud provider API is slow, whether resources are waiting for dependencies, or whether there is a network issue.
I avoid making unnecessary changes just to reduce the execution time.
**Simple ga gurthupettuko:**
Apply slow ayithe → ye resource ekkuva time tiskuntundo chudu.
Provider API, dependencies, and network check chesi → actual reason fix cheyyi.
**Eng:** Terraform apply is slow → find which resource is taking time → check provider API, dependencies, and network → fix the actual reason.
**Example:**
terraform apply → output lo ye resource daggara time ekkuva avutundo chudu.
Cloud Console → aa resource creation status check cheyyi.
Terraform output → dependency wait chesthunda verify cheyyi.
Network/API issue → provider response slow ga unda check cheyyi.
Actual reason fix chesi → apply malli run/continue cheyyi.

---

**22. Terraform resources have dependencies. How do you handle them?**
Terraform normally understands dependencies from resource references.
If Terraform cannot understand a required dependency automatically, I can define an explicit dependency when necessary.
I avoid adding unnecessary dependencies because they can make the execution order more restrictive.
**Simple ga gurthupettuko:**
Resource A mundu create avvali, B tarvatha avvali ante → Terraform references tho dependency understand chestundi.
Automatic ga understand kakapothe → required place lo explicit dependency add cheyyi.
**Eng:** Resources have an order → Terraform normally detects dependencies automatically → if it cannot detect one, define the required dependency explicitly.
**Example:**
Terraform code → Resource B lo Resource A reference unda chudu.
terraform graph → resource dependency relationship check cheyyi.
Automatic dependency lekapothe → required resource ki depends_on add cheyyi.
terraform plan → correct order lo changes plan avutunnaya verify cheyyi.

**23. Terraform resource keeps changing on every plan. What will you investigate?**

First, I identify which attribute is changing every time.
Then I check whether the value is generated dynamically, changed by the provider, modified outside Terraform, or defined differently in the configuration.
I fix the source of the repeated change instead of ignoring the plan blindly.
**Simple ga gurthupettuko:**
Prathi plan lo same resource change ayithe → exact attribute enti chudu.
Dynamic value, provider change, manual change, or code difference check chesi → permanent fix cheyyi.
**Eng:** A resource changes on every plan → find the exact changing value → check dynamic values, provider behavior, manual changes, and code → fix the repeated change.
**Example:**
terraform plan → prathi sari change avutunna attribute identify cheyyi.
terraform state show RESOURCE_NAME → current state value check cheyyi.
Terraform code → same attribute ela define chesaro chudu.
AWS Console → actual value compare cheyyi.
Dynamic/provider/manual change reason fix chesi → terraform plan malli run cheyyi.

---

**24. Terraform resource was deleted outside Terraform. What will happen?**
Terraform state may still contain the resource even though it no longer exists in the real environment.
During the next plan, Terraform can detect the difference and may propose creating the resource again.
I review the plan carefully before allowing Terraform to recreate it.
**Simple ga gurthupettuko:**
Resource Terraform bayata delete ayithe → state lo old information undachu.
Plan run chesthe difference kanipisthundi → recreate cheyyala vadha decide chesi → appudu apply cheyyi.
**Eng:** A resource was deleted outside Terraform → the state may still contain it → Terraform detects the difference in plan → review it before recreating the resource.
**Example:**
AWS Console → resource actually deleted ayyinda chudu.
terraform state list → state lo resource still unda check cheyyi.
terraform plan → Terraform recreate cheyyali ani chupisthunda chudu.
Resource required ayithe → plan review chesi terraform apply cheyyi.
Required kakapothe → configuration/state correct cheyyi.

---

**25. Terraform state contains a resource that should no longer be managed. What will you do?**
First, I confirm that the real resource should no longer be managed by Terraform.
Then I remove the resource from Terraform state without deleting the actual infrastructure when that is the required outcome.
After that, I run Terraform plan and verify that no unwanted change is planned.
**Simple ga gurthupettuko:**
Resource ni Terraform manage cheyyakudadhu kani actual resource kavali ante → state nunchi remove cheyyi.
Actual resource delete avvakunda → plan run chesi result verify cheyyi.
**Eng:** A resource should no longer be managed → remove it from Terraform state without deleting the real resource → run plan → verify the result.
**Example:**
terraform state show RESOURCE_NAME → resource state lo unda confirm cheyyi.
Actual infrastructure → resource still required aa verify cheyyi.
terraform state rm RESOURCE_NAME → Terraform management nunchi remove cheyyi.
AWS Console → actual resource still unda check cheyyi.
terraform plan → unwanted change ledu ani verify cheyyi.

---

**26. How do you handle Terraform changes in Production?**
I never directly apply an unreviewed Terraform change to Production.
I review the code, run Terraform plan, verify the exact changes, and get the required approval.
For important changes, I also ensure that backups and a recovery plan are available.
**Simple ga gurthupettuko:**
Production change ante → code review cheyyi → plan check cheyyi → changes verify cheyyi.
Approval teesukoni → backup/recovery ready ga petti → apply cheyyi.
**Eng:** Production Terraform change → review the code → check the plan → verify the changes → get approval → apply with a recovery plan.
**Example:**
Git/PR → Terraform code review cheyyi.
terraform plan → exact Production changes chudu.
Backup/recovery plan → required aa ready ga unda check cheyyi.
Approval → required approval vachinda verify cheyyi.
Approval tarvatha → terraform apply → change execute chesi verify cheyyi.

---

**27. Terraform plan shows that many resources will be changed. What will you do?**
I do not apply the entire plan immediately.
First, I identify why so many resources are changing and whether the change is expected.
I check recent code, provider versions, modules, variables, and state before deciding whether to continue.
**Simple ga gurthupettuko:**
Chala resources change ani plan vaste → immediate apply cheyyaku.
Enduku anni change avutunnayo → code, provider, module, variables, state check cheyyi.

**Eng:** Many resources are changing → do not apply immediately → find why they are changing → check code, provider, modules, variables, and state.
**Example:**
terraform plan → enni resources change avutunnayo chudu.
Plan output → common change pattern unda identify cheyyi.
Git → recent code changes check cheyyi.
Provider/module versions → recent update unda verify cheyyi.
Variables/state → unexpected values unnaya chudu.
Reason confirm ayyaka → plan malli run chesi → appudu apply decision teesukovali.

---

**28. Terraform state has sensitive information. How will you secure it?**
I treat Terraform state as sensitive because it can contain confidential resource information.
I use a secure remote backend, restrict access, enable encryption where supported, and avoid exposing the state through logs or source control.
Only the required users and pipelines should have access.
**Simple ga gurthupettuko:**
Terraform state lo sensitive information undachu → secure remote backend lo store cheyyi → access restrict cheyyi.
Encryption and proper permissions use chesi → unnecessary access avoid cheyyi.
**Eng:** Terraform state can contain sensitive information → store it securely → restrict access → use encryption where supported → allow access only to required users and pipelines.
**Example:**
AWS S3 backend → state file secure bucket lo unda check cheyyi.
S3 settings → encryption enabled aa verify cheyyi.
IAM → only required users/pipeline ki access unda check cheyyi.
Git repository → .tfstate accidentally commit ayyinda chudu.
Pipeline logs → state/sensitive values expose avutunnaya verify cheyyi.

---

**29. Terraform module version changed and Production infrastructure changed unexpectedly. What will you do?**
First, I compare the current module version with the previous Production version.
Then I identify which module change caused the infrastructure difference and review the Terraform plan.
If the change is not expected, I restore the known-good module version and test it before applying again.
**Simple ga gurthupettuko:**
Module version change tarvatha Prod change ayithe → old and new module versions compare cheyyi.
Problematic change identify chesi → required ayithe old working version ki return ayi → test chesi apply cheyyi.
**Eng:** Module update changed Production → compare the old and new module versions → identify the bad change → restore the known-good version if required → test before applying.
**Example:**
Git → current module version and previous Production version compare cheyyi.
terraform plan → module change valla ye resources change avutunnayo chudu.
Module code → problematic change identify cheyyi.
Known-good version ki return ayi → terraform plan → changes correct ga unnaya verify cheyyi.
Approval tarvatha → terraform apply cheyyi.

---

**30. Terraform pipeline failed after code was merged. What will you check?**
First, I check the pipeline logs and identify whether the failure happened during init, plan, or apply.
Then I check the merged Terraform code, provider versions, variables, credentials, backend access, and recent changes.
After fixing the root cause, I run the validation and plan again before applying.
**Simple ga gurthupettuko:**
Terraform pipeline fail ayithe → init/plan/apply lo ekkada fail ayindo chudu.
Merged code, provider, variables, credentials, backend check chesi → issue fix chesi → plan malli run cheyyi.
**Eng:** Terraform pipeline failed → find whether init, plan, or apply failed → check the merged code and environment → fix the issue → run the plan again.
**Example:**
Jenkins pipeline logs → init, plan, or apply lo ekkada fail ayyindo chudu.
Merged Git code → recent change check cheyyi.
Provider/variables/credentials → correct ga unnaya verify cheyyi.
Backend → accessible ga unda check cheyyi.
Issue fix chesi → pipeline validation → terraform plan malli run cheyyi.

---

**31. Terraform apply succeeded, but the application is not working. What will you check?**
A successful Terraform apply only confirms that the infrastructure changes were completed.
I check the application, networking, security rules, load balancer, DNS, compute resources, configuration, and dependent services.
I also verify whether Terraform created the infrastructure with the expected values.
**Simple ga gurthupettuko:**
Terraform apply success ante application kuda work avutundi ani kaadu.
App, network, security, load balancer, DNS, configuration check chesi → actual issue identify cheyyali.
**Eng:** Terraform apply succeeded but the application is not working → check the application, network, security, load balancer, DNS, and configuration → find the actual issue.
**Example:**
terraform apply → infrastructure successfully create/update ayyinda chudu.
AWS Console → compute resource running unda check cheyyi.

Security Group → required ports allow unnaya verify cheyyi.
Load Balancer → target healthy ga unda chudu.
DNS → correct endpoint ki resolve avutunda check cheyyi.
Application logs → actual application issue identify cheyyi.

---

**32. How do you prevent accidental Terraform changes in Production?**
I use separate Production state and controlled access to the Production environment.
I require code review, Terraform plan review, and approval before applying important changes.
I also use CI/CD controls so that only authorized pipelines or users can modify Production.
**Simple ga gurthupettuko:**
Production accidental change avoid cheyyali ante → separate state and restricted access use cheyyi.
Code review → plan review → approval tarvatha matrame apply cheyyi.
**Eng:** Prevent accidental Production changes → use separate state and restricted access → review the code and plan → get approval before applying.
**Example:**
Terraform backend → Production ki separate state unda check cheyyi.
IAM → Production access restricted aa verify cheyyi.
Git PR → code review required aa chudu.
terraform plan → Production changes review cheyyi.
CI/CD → approval lekunda apply run avvakunda check cheyyi.

---

**33. How do you manage Terraform across DEV, QA, UAT, and PROD?**
I keep the common infrastructure logic reusable and maintain environment-specific configuration separately.
Each environment should have controlled state and appropriate variables.
Production should have stronger access control and approval requirements than lower environments.
**Simple ga gurthupettuko:**
DEV → QA → UAT → PROD ki common Terraform logic use cheyyi → environment values separate ga maintain cheyyi.
Each environment state separate ga petti → Prod ki extra approval and access control ivvali.
**Eng:** DEV, QA, UAT, and PROD use common Terraform logic → keep environment values and state separate → apply stronger controls to Production.
**Example:**
Terraform module → common infrastructure logic maintain cheyyi.
DEV variables → DEV values separate ga petti plan run cheyyi.
QA/UAT → respective variables and state use cheyyi.
PROD → separate state and restricted access use cheyyi.
Production → approval tarvatha matrame terraform apply cheyyi.

---

**34. Terraform configuration is becoming difficult to maintain. What will you do?**
First, I identify repeated code, very large files, and tightly coupled resources.
Then I organize the configuration into reusable modules and separate related infrastructure logically.
I also standardize variables, naming, and outputs so the configuration is easier for the team to understand.
**Simple ga gurthupettuko:**
Terraform code messy ayithe → repeated code and large files identify cheyyi.
Modules ga divide chesi → naming and variables standardize chesi → easy ga maintain cheyyi.
**Eng:** Terraform code is difficult to maintain → find repeated and tightly coupled code → organize it into modules → standardize variables and naming.
**Example:**
Terraform files → repeated code and large files identify cheyyi.
Related resources → separate modules ga organize cheyyi.
Variables → common naming and structure use cheyyi.
Outputs → required values standardize cheyyi.
terraform fmt → code formatting correct cheyyi.
terraform validate → configuration valid ga unda verify cheyyi.

---

**35. How would you design a Production-grade Terraform setup?**
I would use reusable modules, remote state, state locking, secure credentials, separate environments, and controlled Production access. Terraform changes would go through code review, automated validation, plan review, and approval before apply.
I would also maintain backups, version control, monitoring, and a recovery process for important infrastructure.
**Simple ga gurthupettuko:**
Production Terraform ante → reusable modules → remote state → secure access → separate environments.
Code review → plan → approval → apply → backup and recovery ready ga undali.
**Eng:** Production Terraform → use reusable modules → secure shared state → separate environments → review the code and plan → get approval → apply safely → keep backup and recovery ready.
**Example:**
Terraform modules → reusable infrastructure code maintain cheyyi.
Remote backend → shared state and locking configure cheyyi.
DEV/QA/UAT/PROD → separate environment configuration and state use cheyyi.
CI/CD → terraform fmt, terraform validate, terraform plan run cheyyi.
Code review → plan review → approval complete cheyyi.
Approval tarvatha → terraform apply → infrastructure verify cheyyi.
Backups/recovery → important infrastructure ki recovery plan ready ga maintain cheyyi